Extracting Domain Specific Parts from General Purpose Programs

Matei Budiu

University of Illinois Urbana-Champaign,
Illinois, USA,
mbudiu2@illinois.edu

Bhavya Hirani

University of Illinois Urbana-Champaign,
Illinois, USA,
bhirani2@illinois.edu

Charith Mendis

University of Illinois Urbana-Champaign,
Illinois, USA,
charithm@illinois.edu

Optimizing the matmul: One kernel to rule them all!

```
FOR I = 1, N:
  FOR J = 1, N:
    FOR K = 1, N:
      C[I, J] += A[I, K] * B[K, J]
    ENDFOR
  ENDFOR
ENDFOR
```

Optimizing the matmul: One kernel to rule them all!

```
FOR I = 1, N:                  ----- Dim 0
  FOR J = 1, N:                ----- Dim 1
    FOR K = 1, N:              ----- Dim 2
      C[I, J] += A[I, K] * B[K, J]
    ENDFOR
  ENDFOR
ENDFOR
```

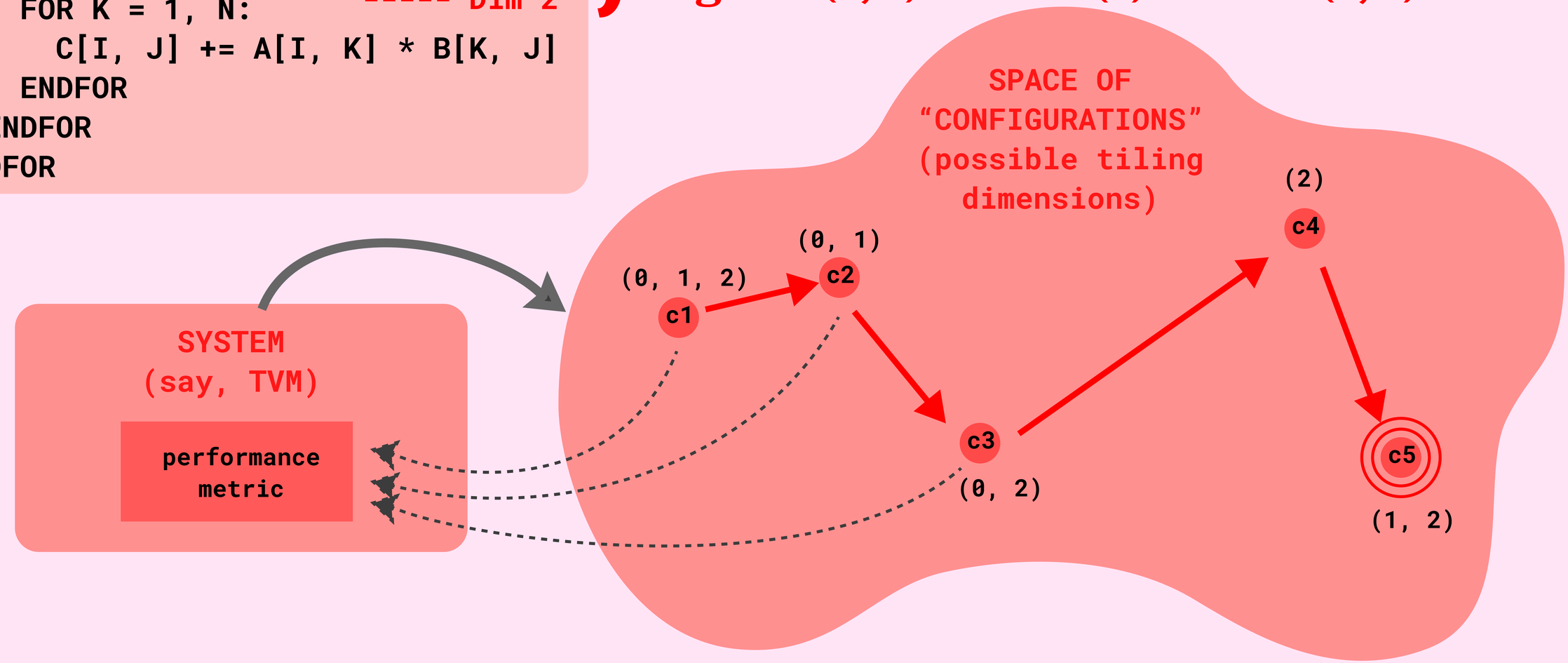
}

Different tiling combos possible
eg. tile (0, 1) OR tile (0) OR tile (1, 0)

Optimizing the matmul: One kernel to rule them all!

```
FOR I = 1, N:                  ----- Dim 0
  FOR J = 1, N:                ----- Dim 1
    FOR K = 1, N:              ----- Dim 2
      C[I, J] += A[I, K] * B[K, J]
    ENDFOR
  ENDFOR
ENDFOR
```

Different tiling combos possible
eg. tile (0, 1) OR tile (0) OR tile (1, 0)



And that’s not it!

Domain specific languages try to specialize for heavily-used operations in that domain so that they’re fast. Some optimizations include

- Reductions and scans
- Layout optimization
- Graph-level rewrites

And that's not it!

Domain specific languages try to specialize for heavily-used operations in that domain so that they're fast. Some optimizations include

- **Reductions and scans**
- **Layout optimization**
- **Graph-level rewrites**

However, they are hard to get used to and write code in, and are quite rigid.

```
// Are you ready? We're going to use all of the features abo
Func gradient_fast("gradient_fast");
gradient_fast(x, y) = x + y;

// We'll process 64x64 tiles in parallel.
Var x_outer, y_outer, x_inner, y_inner, tile_index;
gradient_fast
    .tile(x, y, x_outer, y_outer, x_inner, y_inner, 64, 64)
    .fuse(x_outer, y_outer, tile_index)
    .parallel(tile_index);
```

```
@tvm.script.ir_module
class MyModule:
    @T.prim_func
    def mm_relu(A: T.Buffer((128, 128), "float32"),
                B: T.Buffer((128, 128), "float32"),
                C: T.Buffer((128, 128), "float32")):
        Y = T.alloc_buffer((128, 128), dtype="float32")
        for i, j, k in T.grid(128, 128, 128):
            with T.sblock("Y"):
                vi = T.axis.spatial(128, i)
                vj = T.axis.spatial(128, j)
                vk = T.axis.reduce(128, k)
                with T.init():
                    Y[vi, vj] = T.float32(0)
                Y[vi, vj] = Y[vi, vj] + A[vi, vk] * B[vk, vj]
    for i, j in T.grid(128, 128):
        with T.sblock("C"):
            vi = T.axis.spatial(128, i)
            vj = T.axis.spatial(128, j)
            C[vi, vj] = T.max(Y[vi, vj], T.float32(0))
```

Our proposed solution

Extract domain specific parts from general purpose programs

Then use the domain specific compiler to optimize those parts!

Why is this a hard problem?
Why not replace these with hand-tuned libs?

```
FOR I = 1, N:
  FOR J = 1, N:
    FOR K = 1, N:
      C[I, J] += A[I, K] * B[K, J]
    ENDFOR
  ENDFOR
ENDFOR
```

Extract domain specific parts from general purpose programs

Then use the domain specific compiler to optimize those parts!

Why is this a hard problem?
Why not replace these with hand-tuned libs?

```
FOR I = 1, N:
  Arr[I] = Arr[I]+1
  FOR J = i, N:
    FOR K = 1, N:
      // other code
      C[I, J] += A[I, K] * B[K, J]
      T[i] = C[I, J]
    ENDFOR
    // more code
  ENDFOR
ENDFOR
```

The major challenge is extracting hidden (latent) representation, which programmers can't do manually

Extract domain specific parts from general purpose programs

Then use the domain specific compiler to optimize those parts!